

02_backend-Dockerfile

#docker

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
├── backend/  
│   └── Dockerfile
```

El archivo `Dockerfile` se encuentra dentro de la carpeta `backend/` del proyecto.

Su función es definir cómo se construye la imagen Docker del backend del laboratorio SIEM MVP. Es decir, indica a Docker qué sistema base debe usar, dónde colocar el código, cómo instalar las dependencias y qué comando debe ejecutarse para arrancar la API FastAPI.

Este archivo se utiliza desde `docker/compose.yml`, concretamente en el servicio `api`:

```
api:  
  build:  
    context: ../backend  
    dockerfile: Dockerfile
```

Esto significa que Docker Compose construye la imagen de la API usando como contexto la carpeta `backend/` y como archivo de construcción `backend/Dockerfile`.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,220p' backend/Dockerfile
```

Este comando muestra el contenido del archivo `backend/Dockerfile`.

Desglose:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, usado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima automáticamente todo el archivo.

```
'1,220p'
```

Indica que se impriman las líneas de la 1 a la 220.

```
backend/Dockerfile
```

Ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
FROM python:3.12-slim
```

```
WORKDIR /app

COPY requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

COPY . /app

CMD ["uvicorn", "app.main:app", "--reload", "--host", "0.0.0.0", "--port", "8000"]
```

4 Función general del archivo

El `Dockerfile` define la imagen Docker del backend.

Mientras que `docker/compose.yml` define los servicios del laboratorio, el `Dockerfile` explica cómo se construye internamente el contenedor de la API.

En este caso, el archivo realiza estas acciones:

1. Parte de una imagen base con Python 3.12.
2. Define `/app` como directorio de trabajo.
3. Copia el archivo `requirements.txt`.
4. Instala las dependencias Python.
5. Copia el código del backend.
6. Define el comando por defecto para arrancar FastAPI con Uvicorn.

El resultado final es una imagen capaz de ejecutar el backend del laboratorio SIEM.

5 Estructura general del Dockerfile

El archivo tiene cinco bloques principales:

```
FROM python:3.12-slim
```

Define la imagen base.

```
WORKDIR /app
```

Define el directorio de trabajo dentro del contenedor.

```
COPY requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r requirements.txt
```

Copia e instala las dependencias.

```
COPY . /app
```

Copia el código fuente del backend.

```
CMD ["uvicorn", "app.main:app", "--reload", "--host", "0.0.0.0", "--port", "8000"]
```

Define el comando de arranque por defecto.

6 Análisis línea por línea

Imagen base

```
FROM python:3.12-slim
```

La instrucción `FROM` indica la imagen base sobre la que se construirá la nueva imagen Docker.

En este caso:

```
python:3.12-slim
```

se divide en dos partes:

<code>python</code>	→ nombre de la imagen base
<code>3.12-slim</code>	→ versión o etiqueta de la imagen

`python` indica que se parte de una imagen oficial preparada para ejecutar aplicaciones Python.

`3.12` indica la versión de Python utilizada.

`slim` indica que se usa una variante más ligera de la imagen. Esta versión incluye lo necesario para ejecutar Python, pero evita muchos paquetes adicionales que sí estarían en una imagen más completa.

La ventaja de usar `python:3.12-slim` es que la imagen final ocupa menos espacio y contiene menos componentes innecesarios.

En este proyecto tiene sentido porque el backend solo necesita ejecutar una API con FastAPI, conectarse a PostgreSQL y usar las dependencias definidas en `requirements.txt`.

Directorio de trabajo

```
WORKDIR /app
```

La instrucción `WORKDIR` define el directorio de trabajo dentro del contenedor.

A partir de esta línea, las instrucciones posteriores se ejecutarán tomando `/app` como referencia.

Esto significa que comandos como:

```
RUN pip install --no-cache-dir -r requirements.txt
```

se ejecutarán desde `/app`.

Si el directorio `/app` no existe, Docker lo crea automáticamente.

En este proyecto, `/app` representa el lugar donde se ubicará el código del backend dentro del contenedor.

La equivalencia conceptual sería:

Host:	
	<code>siem-lab/backend/</code>
Contenedor:	
	<code>app/</code>

Copia del archivo de dependencias

```
COPY requirements.txt /app/requirements.txt
```

La instrucción `COPY` copia archivos desde el sistema anfitrión hacia la imagen Docker.

Su estructura es:

<code>COPY</code>	origen	destino
-------------------	--------	---------

En este caso:

```
origen → requirements.txt
destino → /app/requirements.txt
```

Como el contexto de construcción definido en `compose.yml` es:

```
context: ../backend
```

Docker interpreta `requirements.txt` como:

```
siem-lab/backend/requirements.txt
```

y lo copia dentro de la imagen en:

```
/app/requirements.txt
```

Este archivo contiene las dependencias Python necesarias para ejecutar el backend:

```
fastapi
uvicorn
sqlalchemy>=2.0
psycopg[binary]
alembic
pytest
httpx
```

Se copia primero `requirements.txt` antes que el resto del código por una razón práctica: aprovechar la caché de Docker.

Si las dependencias no cambian, Docker puede reutilizar la capa donde ya fueron instaladas, haciendo que futuras reconstrucciones sean más rápidas.

Instalación de dependencias

```
RUN pip install --no-cache-dir -r requirements.txt
```

La instrucción `RUN` ejecuta un comando durante la construcción de la imagen.

En este caso ejecuta:

```
pip install --no-cache-dir -r requirements.txt
```

Desglose:

```
pip
```

Es el gestor de paquetes de Python.

```
install
```

Indica que se van a instalar paquetes.

```
--no-cache-dir
```

Evita que `pip` guarde archivos temporales de caché dentro de la imagen.

Esto ayuda a reducir el tamaño final de la imagen Docker.

```
-r requirements.txt
```

La opción `-r` indica que `pip` debe leer la lista de paquetes desde un archivo.

En este caso, el archivo es:

```
requirements.txt
```

Como anteriormente se definió:

```
WORKDIR /app
```

el comando busca el archivo en:

```
/app/requirements.txt
```

Este paso instala dentro de la imagen todas las dependencias necesarias para que el backend pueda arrancar.

Copia del código fuente

```
COPY . /app
```

Esta instrucción copia todo el contenido del contexto de construcción dentro del directorio `/app` de la imagen.

La estructura es:

```
COPY origen destino
```

En este caso:

```
origen  → .  
destino → /app
```

El punto `.` representa el directorio actual del contexto de construcción.

Como el contexto es:

```
context: ../backend
```

el punto `.` equivale a:

```
siem-lab/backend/
```

Por tanto, esta línea copia el contenido de `backend/` dentro de `/app`.

Esto incluye archivos como:

```
app/  
alembic/  
alembic.ini  
Dockerfile  
pytest.ini  
requirements.txt  
tests/
```

En el contenedor, quedarían disponibles en:

```
/app/app/  
/app/alembic/  
/app/alembic.ini  
/app/pytest.ini  
/app/requirements.txt  
/app/tests/
```

Esta línea es la que introduce realmente el código del backend en la imagen.

Comando de arranque por defecto

```
CMD ["uvicorn", "app.main:app", "--reload", "--host", "0.0.0.0", "--port", "8000"]
```

La instrucción `CMD` define el comando por defecto que se ejecutará cuando arranque un contenedor basado en esta imagen.

En este caso, el comando arranca la API FastAPI mediante Uvicorn.

La sintaxis usada es formato JSON array:

```
CMD ["comando", "argumento1", "argumento2"]
```

Este formato es recomendable porque evita problemas de interpretación de shell.

Parte `uvicorn`

```
uvicorn
```

`uvicorn` es el servidor ASGI encargado de ejecutar la aplicación FastAPI.

FastAPI define la aplicación, rutas y lógica, pero necesita un servidor que escuche peticiones HTTP y las entregue a la aplicación.

Ese papel lo cumple Uvicorn.

Parte `app.main:app`

```
app.main:app
```

Esta parte indica a Uvicorn dónde se encuentra la aplicación FastAPI.

Se divide así:

```
app      → paquete o carpeta Python `app/`  
main     → archivo `main.py`  
app      → variable `app` dentro de `main.py`
```

Por tanto, Uvicorn buscará este archivo:

```
/app/app/main.py
```

Y dentro de él buscará una variable llamada:

```
app
```

Normalmente esa variable se define con una instrucción similar a:

```
app = FastAPI()
```

Esto significa que el contenedor arrancará la aplicación definida en `backend/app/main.py`.

Parte `--reload`

```
--reload
```

Activa la recarga automática del servidor cuando detecta cambios en el código.

Es útil durante el desarrollo porque evita tener que reiniciar manualmente el contenedor cada vez que se modifica un archivo.

Sin embargo, en producción normalmente no se usa `--reload`, porque consume más recursos y está pensado para desarrollo.

En este laboratorio tiene sentido porque el proyecto se ha construido como entorno de desarrollo y aprendizaje.

Parte `--host`

```
--host
```

Indica la dirección de red en la que escuchará Uvicorn.

Va acompañada del valor:

```
0.0.0.0
```

Parte `0.0.0.0`

```
0.0.0.0
```

Significa que Uvicorn escuchará en todas las interfaces de red disponibles dentro del contenedor.

Esto es importante en Docker.

Si la API escuchara solo en:

```
127.0.0.1
```

dentro del contenedor, podría quedar accesible únicamente desde el propio contenedor y no desde el host.

Con:

```
0.0.0.0
```

Docker puede redirigir correctamente las peticiones desde el puerto publicado en el host hacia el puerto interno del contenedor.

Parte `--port`

```
--port
```

Indica el puerto donde escuchará Uvicorn.

Va acompañado del valor:

```
8000
```

Parte `8000`

```
8000
```

Es el puerto interno del contenedor donde se ejecuta la API.

En `docker/compose.yml`, este puerto se publica hacia el host mediante:

```
ports:
  - "127.0.0.1:${API_PORT:-8000}:8000"
```

Esto significa que el puerto interno `8000` del contenedor se expone en el puerto local `8000`, salvo que `API_PORT` indique otro valor.

7 Relación entre Dockerfile y docker/compose.yml

El `Dockerfile` no se ejecuta directamente por sí solo durante el uso normal del laboratorio. Lo invoca Docker Compose desde el servicio `api`.

En `docker/compose.yml` aparece:

```
api:
  build:
    context: ../backend
    dockerfile: Dockerfile
```

Esto significa:

1. Docker Compose entra en la carpeta `backend/`.
2. Busca el archivo `Dockerfile`.
3. Construye una imagen para el backend.
4. Crea un contenedor llamado `siem-api`.
5. Ejecuta la API FastAPI dentro de ese contenedor.

También hay que tener en cuenta esta línea del `compose.yml`:

```
command: uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Cuando `docker/compose.yml` define `command`, ese comando sobrescribe el `CMD` definido en el `Dockerfile`.

Por tanto, en este proyecto hay una duplicación controlada:

```
Dockerfile CMD → comando por defecto de la imagen
compose.yml command → comando usado cuando se levanta con Docker Compose
```

En la práctica, al usar Docker Compose, se ejecuta el `command` del `compose.yml`.

8 Relación con el flujo técnico del laboratorio

El `Dockerfile` participa en el flujo de arranque del laboratorio de esta forma:

```
docker/compose.yml
  ↓
servicio api
  ↓
build context ../backend
  ↓
backend/Dockerfile
  ↓
imagen Python 3.12 + dependencias
  ↓
contenedor siem-api
  ↓
uvicorn app.main:app
  ↓
backend/app/main.py
  ↓
API FastAPI disponible
```

Este archivo no procesa eventos, reglas ni alertas directamente, pero prepara el entorno necesario para que el backend pueda hacerlo.

Sin el `Dockerfile`, habría que instalar Python, dependencias y ejecutar Uvicorn manualmente en la máquina anfitriona.

9 Errores típicos relacionados

♦ Error al instalar dependencias

Si falla esta línea:

```
RUN pip install --no-cache-dir -r requirements.txt
```

puede deberse a:

- Error de red al descargar paquetes.
- Paquete mal escrito en `requirements.txt`.
- Versión incompatible de alguna dependencia.
- Falta de librerías del sistema necesarias para compilar un paquete.

En este proyecto, el uso de:

```
psycopg[binary]
```

ayuda a evitar problemas de compilación del driver de PostgreSQL, porque instala una versión binaria preparada.

♦ Error `ModuleNotFoundError`

Puede aparecer si Uvicorn no encuentra:

```
app.main
```

Esto puede deberse a:

- El código no está copiado correctamente en `/app`.
- El directorio de trabajo no es correcto.
- Falta el archivo `backend/app/main.py`.
- Falta el archivo `backend/app/__init__.py`.

En este proyecto, `WORKDIR /app` y `COPY . /app` hacen que `app/main.py` quede accesible como módulo Python.

♦ La API no se ve desde el navegador

Si Uvicorn escucha en:

```
127.0.0.1
```

dentro del contenedor, puede no ser accesible desde fuera del contenedor.

Por eso se usa:

```
--host 0.0.0.0
```

Esta configuración permite que Docker publique correctamente el servicio hacia el host.

♦ Cambios en código no aplican

El `Dockerfile` copia el código durante la construcción de la imagen:

```
COPY . /app
```

Pero en desarrollo, `docker/compose.yml` monta el código local:

```
volumes:
- ../backend:/app
```

Esto significa que el contenido montado desde el host puede sobrescribir lo que se copió en la imagen.

Por eso los cambios deberían reflejarse si Uvicorn está arrancado con:

```
--reload
```

10 Comandos útiles relacionados

Construir la imagen desde Docker Compose:

```
docker compose --env-file docker/.env -f docker/compose.yml build api
```

Levantar solo la API:

```
docker compose --env-file docker/.env -f docker/compose.yml up -d api
```

Ver logs de la API:

```
docker logs siem-api
```

Entrar dentro del contenedor de la API:

```
docker exec -it siem-api bash
```

Ver archivos dentro del contenedor:

```
docker exec -it siem-api ls -la /app
```

Comprobar versión de Python dentro del contenedor:

```
docker exec -it siem-api python --version
```

Comprobar dependencias instaladas:

```
docker exec -it siem-api pip list
```

Comprobar si Uvicorn está disponible:

```
docker exec -it siem-api uvicorn --version
```

11 Cómo explicarlo en la presentación

El archivo `backend/Dockerfile` define cómo se construye la imagen del backend del laboratorio. Parte de una imagen ligera de Python 3.12, establece `/app` como directorio de trabajo, copia el archivo `requirements.txt`, instala las dependencias necesarias y después copia el código fuente del backend.

Finalmente, define como comando por defecto el arranque de la API mediante Uvicorn, apuntando a `app.main:app`, que es la aplicación FastAPI definida en el archivo `backend/app/main.py`.

Este Dockerfile permite que el backend pueda ejecutarse dentro de un contenedor de forma reproducible, sin depender de una instalación manual de Python y librerías en la máquina anfitriona. Además, combinado con Docker Compose, facilita levantar la API junto con PostgreSQL y Adminer en un entorno controlado.